

Sortieren | `Comparator<T>`

Kategorie	Details
Definition	<code>Comparator<T></code> ist ein funktionales Interface im Paket <code>java.util</code> , das eine benutzerdefinierte Vergleichslogik für Objekte des generischen Typs <code>T</code> bereitstellt. Es erweitert nicht <code>Comparable<T></code> , sondern bietet eine externe Vergleichsstrategie.
Funktionale Methode (SAM)	<code>int compare(T o1, T o2)</code> ; Gibt eine Ganzzahl zurück: $\rightarrow$ negativ (< 0), wenn $o1 < o2 \rightarrow 0$, wenn $o1 == o2 \rightarrow$ positiv (> 0), wenn $o1 > o2$
Zusätzliche Standardmethoden (Java 8+ Default Methods)	<code>Comparator<T> reversed()</code> $\rightarrow$ Gibt einen umgekehrten <code>Comparator</code> zurück. <code>Comparator<T> thenComparing(Comparator<? super T> other)</code> $\rightarrow$ Verknüpft mehrere Vergleiche. <code>Comparator<T> thenComparingInt(ToIntFunction<? super T> keyExtractor)</code> , <code>thenComparingDouble(ToDoubleFunction<? super T> keyExtractor)</code> , <code>thenComparingLong(ToLongFunction<? super T> keyExtractor)</code> $\rightarrow$ Optimierte Methoden für primitive Werte (vermeidet Autoboxing).
Factory-Methoden (static)	<code>Comparator<T> comparing(Function<? super T, ? extends U> keyExtractor)</code> $\rightarrow$ Erstellt einen <code>Comparator</code> basierend auf einem Schlüsselselektor. <code>Comparator<T> comparingInt(ToIntFunction<? super T> keyExtractor)</code> , <code>comparingLong(ToLongFunction<? super T> keyExtractor)</code> , <code>comparingDouble(ToDoubleFunction<? super T> keyExtractor)</code> $\rightarrow$ Effiziente Vergleiche für primitive Typen.
Sortierungsoptionen	<code>Comparator<T> naturalOrder()</code> $\rightarrow$ Gibt einen <code>Comparator</code> zurück, der die natürliche Reihenfolge (<code>Comparable<T></code>) verwendet. <code>Comparator<T> reverseOrder()</code> $\rightarrow$ Kehrt die natürliche Reihenfolge um. <code>Comparator<T> nullsFirst(Comparator<? super T> c)</code> $\rightarrow$ Behandelt <code>null</code> -Werte zuerst. <code>Comparator<T> nullsLast(Comparator<? super T> c)</code> $\rightarrow$ Behandelt <code>null</code> -Werte zuletzt.
Mehrstufige Sortierung	<code>thenComparing(...)</code> ermöglicht eine kaskadierende Sortierung nach mehreren Attributen. Falls der erste Vergleich <code>0</code> zurückgibt, wird das nächste Kriterium geprüft. Unterstützt primitiv-optimierte Varianten für bessere Performance.
Nutzung in <code>Collection.sort()</code> und Streams	<code>Collections.sort(List<T> list, Comparator<? super T> c)</code> ; <code>list.sort(Comparator<? super T> c)</code> ; <code>Stream<T>.sorted(Comparator<? super T> comparator)</code> ;
Nutzung in <code>TreeSet</code> und <code>TreeMap</code>	<code>TreeSet<T></code> kann mit einem <code>Comparator<T></code> initialisiert werden (<code>new TreeSet<>(comparator)</code>). <code>TreeMap<K, V></code> kann mit einem <code>Comparator<K></code> initialisiert werden (<code>new TreeMap<>(comparator)</code>).

Kategorie	Details
Leistungsaspekte	Methoden wie <code>comparingInt()</code> , <code>comparingLong()</code> und <code>comparingDouble()</code> sind effizienter als <code>comparing()</code> mit Autoboxing. <code>Stream.sorted(Comparator)</code> erfordert Materialisierung der Elemente und ist speicherintensiv. <code>Comparator</code> kann in parallelen Streams ineffizient sein, wenn die Vergleichsoperation teuer ist.
Einsatz in parallelen Sortierungen	<code>Arrays.parallelSort(T[] a, Comparator<? super T> c);</code> nutzt Multithreading für die Sortierung.
Typische Fehlerquellen	<code>compare()</code> muss konsistent mit <code>equals()</code> sein, wenn er in <code>SortedSet</code> oder <code>SortedMap</code> verwendet wird. Asymmetrische Implementierungen (z. B. <code>compare(o1, o2) != -compare(o2, o1)</code>) führen zu unerwartetem Verhalten. <code>Comparator</code> sollte transitiv sein (<code>a < b</code> und <code>b < c</code> impliziert <code>a < c</code>). Nullwerte müssen explizit behandelt werden (<code>nullsFirst()</code> oder <code>nullsLast()</code>).
Java-Versionen & Entwicklungen	Seit Java 8: Einführung von <code>Default Methods</code> und <code>Factory Methods</code> (<code>comparing()</code> , <code>thenComparing()</code>). Seit Java 9: Verbesserte API-Dokumentation mit zusätzlichen Beispielen.